Weighted Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

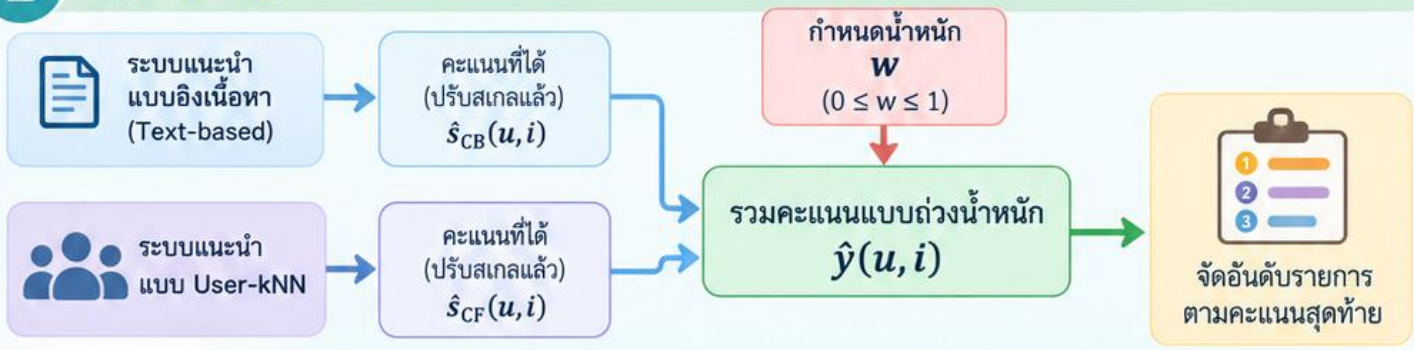
หนึ่งใน 7 รูปแบบของ Burke

1 Weighted Hybrid คืออะไร?

Weighted Hybrid เป็นวิธีผสานผลลัพธ์จากตัวแนะนำหลายระบบ โดยกำหนด **"น้ำหนัก (weight)"** ให้กับแต่ละระบบ แล้วนำคะแนนที่ได้มารวมกัน เพื่อให้ได้คะแนนสุดท้ายสำหรับการแนะนำรายการ

ในบทนี้ใช้ตัวอย่างการผสานระหว่าง ระบบแนะนำที่อิงข้อมูลเนื้อหา (Text-based Recommender) กับ ระบบ User-kNN และจัดเป็นแบบ **Late Fusion** (รวมคะแนนในขั้นตอนท้าย)

2 แนวคิดหลัก



3 สูตรสำคัญ

$$\hat{y}(u,i) = w \cdot \hat{s}_{CB}(u,i) + (1-w) \cdot \hat{s}_{CF}(u,i)$$
$$0 \le w \le 1$$

ควรปรับสเกลคะแนนของแต่ละระบบให้อยู่ในช่วงเดียวกัน เช่น 0–1 ก่อนนำมารวมกัน เพื่อให้การถ่วงน้ำหนักมีความหมาย

- $\hat{y}(u,i)$ = คะแนนแนะนำสุดท้ายของผู้ใช้ u สำหรับรายการ i
- $\hat{s}_{CB}(u,i)$ = คะแนนจากระบบอิงเนื้อหา (หลังปรับสเกลแล้ว)
- $\hat{s}_{CF}(u,i)$ = คะแนนจากระบบ User-kNN (หลังปรับสเกลแล้ว)
- w = น้ำหนักของระบบอิงเนื้อหา (อีกส่วน 1 – w เป็นของ User-kNN)

4 ตัวอย่างการคำนวณอย่างง่าย

กำหนดให้ w = 0.70 (ให้น้ำหนักกับระบบอิงเนื้อหา 70% และ User-kNN 30%)
พิจารณารายการแนะนำ 2 รายการ ได้แก่ i2 และ i9

รายการ	$\hat{s}_{CB}(u,i)$	$\hat{s}_{CF}(u,i)$	การคำนวณคะแนนสุดท้าย	$\hat{y}(u,i)$
i2	0.80	0.50	0.70 × 0.80 + 0.30 × 0.50 = 0.56 + 0.15	**0.71**
i9	0.40	0.85	0.70 × 0.40 + 0.30 × 0.85 = 0.28 + 0.255	**0.535**

ผลการจัดอันดับ

อันดับ	รายการ	คะแนนสุดท้าย
1	**i2**	**0.71**
2	i9	0.535

ดังนั้น รายการ i2 ได้รับการจัดอันดับสูงกว่า i9 เพราะ 0.71 > 0.535

5 สรุปสั้น ๆ

จุดเด่น

- เข้าใจง่าย และนำไปใช้งานได้ไม่ซับซ้อน
- สามารถปรับสมดุลจุดแข็งของหลายระบบได้
- ยืดหยุ่นในการกำหนดน้ำหนักตามลักษณะปัญหา
- เป็นวิธีแบบ Late Fusion รวมคะแนนในขั้นตอนท้าย

ข้อควรระวัง

- ควรปรับสเกลคะแนนของแต่ละระบบให้อยู่ในช่วงเดียวกันก่อนนำมารวมกัน
- การเลือกค่าน้ำหนัก w มีผลต่อคุณภาพของการแนะนำ ควรใช้การทดลอง/validation เพื่อหาค่าที่เหมาะสม
- ไม่เหมาะกับกรณีที่คะแนนจากแต่ละระบบมีสเกลต่างกันมากโดยไม่ปรับสเกล

Switching Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

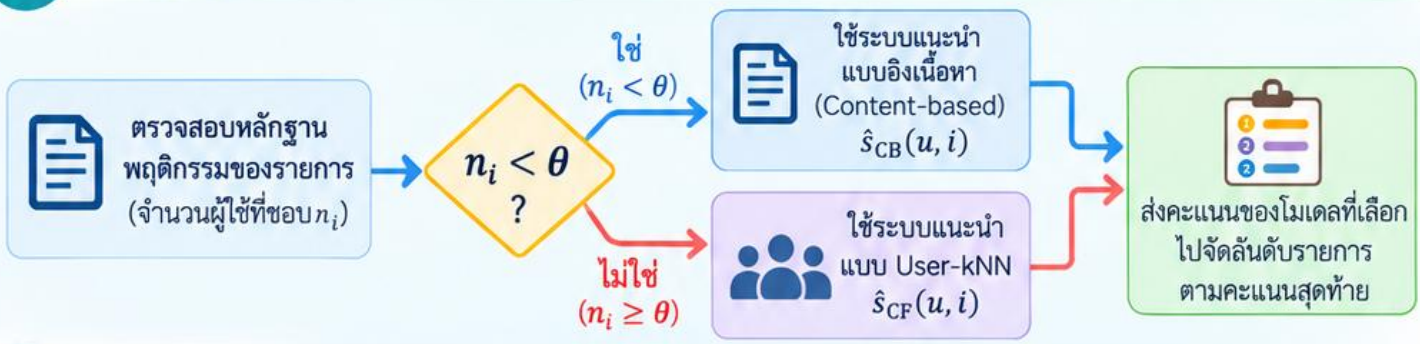
หนึ่งใน 7 รูปแบบของ Burke

1 Switching Hybrid คืออะไร?

Switching Hybrid เป็นวิธีผสานผลลัพธ์ที่ระบบจะเลือกใช้ **"เพียงหนึ่งโมเดล"** ในแต่ละครั้ง โดยพิจารณาจากสถานการณ์หรือคุณลักษณะของข้อมูล เช่น จำนวนผู้ใช้ที่เคยชอบรายการนั้น

ตัวอย่างในบทนี้: ถ้ารายการมีผู้ใช้ที่ชอบน้อยกว่าเกณฑ์ (เช่น น้อยกว่า 2 คน) จะใช้ระบบแนะนำแบบอิงเนื้อหา (Text-based) แต่ถ้ามีผู้ใช้ที่ชอบตั้งแต่เกณฑ์ขึ้นไป จะใช้ระบบ User-kNN

2 แนวคิดหลัก



3 กฎ/สูตรสำคัญ

$$\hat{y}(u,i) = \begin{cases} \hat{s}_{CB}(u,i) & \text{if } n_i < \theta \\ \hat{s}_{CF}(u,i) & \text{if } n_i \geq \theta \end{cases}$$

- $\hat{y}(u,i)$ = คะแนนแนะนำสุดท้ายของผู้ใช้ u สำหรับรายการ i
- n_i = จำนวนผู้ใช้ที่ชอบรายการ i
- θ = เกณฑ์จำนวนผู้ใช้ที่ชอบ (ในตัวอย่างใช้ $\theta = 2$)
- $\hat{s}_{CB}(u,i)$ = คะแนนจากระบบอิงเนื้อหา (Text-based)
- $\hat{s}_{CF}(u,i)$ = คะแนนจากระบบ User-kNN

4 ตัวอย่างการคำนวณอย่างง่าย

กำหนดให้เกณฑ์ $\theta = 2$ (ถ้ารายการมีผู้ใช้ที่ชอนน้อยกว่า 2 คน ใช้ Text-based และถ้ามีตั้งแต่ 2 คนขึ้นไป ใช้ User-kNN)

รายการ	n_i (จำนวนผู้ใช้ที่ชอบ)	คะแนน Text-based $\hat{s}_{CB}(u,i)$	คะแนน User-kNN $\hat{s}_{CF}(u,i)$	โมเดลที่เลือก	คะแนนสุดท้าย $\hat{y}(u,i)$
i2	1	0.82	0.55	Text-based (1 < 2)	**0.82**
i8	4	0.60	0.74	User-kNN (4 ≥ 2)	**0.74**
i10	0	0.61	0.30	Text-based (0 < 2)	**0.61**

ผลการจัดอันดับ

อันดับ	รายการ	คะแนนสุดท้าย
1	**i2**	**0.82**
2	i8	0.74
3	i10	0.61

ดังนั้น รายการ i2 ได้รับการจัดอันดับสูงกว่า i8 และ i10 เพราะ 0.82 > 0.74 > 0.61

5 สรุปสั้น ๆ

จุดเด่น

- แก้ปัญหา cold-start ได้ตรงไปตรงมา
- เข้าใจง่าย เลือกใช้โมเดลที่เหมาะกับสถานการณ์
- เหมาะเมื่อแต่ละโมเดลมีจุดแข็งต่างกันในคนละสถานการณ์

ข้อควรระวัง

- ผลลัพธ์ขึ้นกับเกณฑ์ θ ถ้าเลือกค่าไม่เหมาะสม อาจทำให้คุณภาพการแนะนำไม่ดี
- อาจเปลี่ยนระบบบ่อยเกินไป หากข้อมูลอยู่ใกล้เกณฑ์ และอาจทำให้ผลลัพธ์ไม่สม่ำเสมอ

Mixed Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

หนึ่งใน 7 รูปแบบของ Burke

1 Mixed Hybrid คืออะไร?

Mixed Hybrid เป็นวิธีผสมผลลัพธ์จากหลายระบบ โดยไม่รวมคะแนนเป็นตัวเลขเดียว แต่นำรายการแนะนำจากหลายระบบมาจัดแสดงร่วมกัน ในรูปแบบต่าง ๆ เช่น สลับรายการทีละตัว หรือจัดเป็นหลายแถวบนหน้าแสดงผล (homepage) เพื่อให้ผู้ใช้เห็นรายการที่หลากหลายจากหลายแนวทาง

ช่วยเพิ่มความหลากหลาย (diversity) และเปิดโอกาสให้ผู้ใช้ค้นพบรายการที่น่าสนใจได้จากหลายมุมมอง โดยไม่ต้องตัดสินใจว่าระบบใดสำคัญกว่า

2 แนวคิดหลัก



3 วิธีคิดสำคัญ

1. หยิบรายการลำดับแรกจากระบบที่ 1
2. หยิบรายการลำดับแรกจากระบบที่ 2
3. สลับไปมาทีละรายการ (interleave)
4. หากเจอรายการที่ซ้ำกับรายการที่เคยเลือกแล้ว ให้ข้ามไป
5. ทำต่อไปจนได้จำนวนรายการตามที่ต้องการ

รูปแบบการจัดแสดงผลที่ใช้ได้

- สลับรายการจากแต่ละระบบในลิสต์เดียว
- จัดเป็นหลายแถวบนหน้าแรก เช่น
  - แถวที่ 1: แนะนำจากเนื้อหา (Text-based)
  - แถวที่ 2: แนะนำจากผู้ใช้ที่คล้ายกัน (User-kNN)
- หรือสลับรายการแบบผสมในลิสต์เดียว เพื่อให้ผู้ใช้เห็นความหลากหลาย

4 ตัวอย่างการคำนวณอย่างง่าย

มีรายการแนะนำจาก 2 ระบบ ดังนี้ และสลับหยิบรายการทีละตัว โดยข้ามรายการที่ซ้ำ

รายการจาก Text-based
i5, i11, i12, i2

รายการจาก User-kNN
i9, i7, i8, i2, i10

ผลลัพธ์หลังการสลับรายการ (ข้ามรายการที่ซ้ำ)

อันดับที่	1	2	3	4	5	6	7	8
รายการ	i5	i9	i11	i7	i12	i8	i2	i10

ผลการคำนวณ

รายการ i2 ได้รับการจัดอันดับที่ตำแหน่ง 7
ดังนั้น
Reciprocal Rank (RR) = 1/7 = 0.1429

5 สรุปสั้น ๆ

จุดเด่น

- เพิ่มความหลากหลายของรายการแนะนำ
- เปิดโอกาสให้ผู้ใช้ค้นพบรายการจากหลายแนวทาง
- สามารถจัดแสดงเป็นหลายพื้นที่ (หลายแถว) บนหน้าแรกได้
- เหมาะกับกรณีที่ต้องการความหลากหลายมากกว่าคะแนนเดียว

ข้อควรระวัง

- วัดผลด้วยคะแนนรวมแบบเดียว (เช่น HR@K, MRR) ได้ยาก
- ต้องตัดสินใจจัดสรรพื้นที่ในการแสดงผล
- ตำแหน่งของรายการมีผลต่อการคลิก → เกิด Position Bias
- ลำดับของแถวบนหน้าแรกอาจทำหน้าที่เหมือนน้ำหนักโดยไม่ตั้งใจ

Feature Combination Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

หนึ่งใน 7 รูปแบบของ Burke

1 Feature Combination คืออะไร?

Feature Combination เป็นวิธีผสานข้อมูลหลายแหล่งเข้าด้วยกันตั้งแต่ขั้นตอนนำเข้าข้อมูล (input stage) โดยรวมข้อมูลต่าง ๆ เช่น ข้อมูลของรายการ (item), ข้อมูลเนื้อหา (เช่น ประเภท, ข้อความ), และข้อมูลของผู้ใช้เข้าไปในโมเดลเดียว เพื่อให้โมเดลเรียนรู้จากทุกข้อมูลพร้อมกัน

แนวคิดนี้สอดคล้องกับการรวมคุณลักษณะ (Feature-level combination) และมักเรียกว่า **Early Fusion** โดยตัวอย่างที่ใช้กันอย่างแพร่หลายคือ **LightFM** ซึ่งรวมคุณลักษณะของรายการ (เช่น ข้อความ/ประเภท) กับคุณลักษณะด้านความชอบของผู้ใช้ เข้าไว้ในโมเดลเดียวกัน

2 แนวคิดหลัก

คุณลักษณะของรายการ
- รหัสรายการ (Item ID)
- ประเภท / ข้อความเนื้อหา (เช่น genre, tags, text)

คุณลักษณะของผู้ใช้
- พฤติกรรมความชอบในอดีต
- ข้อมูลโปรไฟล์ (ถ้ามี)

โมเดลเดียว (เช่น LightFM)

นำข้อมูลทุกส่วนเข้าโมเดลพร้อมกันตั้งแต่ขั้นตอนนำเข้า

ให้คะแนนรายการสำหรับผู้ใช้ u $\hat{y}(u,i)$ และจัดลันดับรายการตามคะแนนสูงสุดท้าย

3 สูตรสำคัญ

$$q_i = e_i^{id} + \sum_{g \in G_i} e_g^{genre}, \quad \hat{y}(u,i) = p_u^{\top} q_i$$

คะแนนเกิดจากการคูณเวกเตอร์ของผู้ใช้กับเวกเตอร์ของรายการที่รวมคุณลักษณะทุกส่วนไว้แล้ว (ผลคูณจุด / dot product)

q_i	=	เวกเตอร์ของรายการ i (รวมทุกคุณลักษณะ)
e_i^{id}	=	เวกเตอร์รหัสของรายการ i
e_g^{genre}	=	เวกเตอร์ของแนว g (เช่น ประเภท / ข้อความ)
G_i	=	เซตของแนวภาพยนตร์ของรายการ i
p_u	=	เวกเตอร์ของผู้ใช้ u
$\hat{y}(u,i)$	=	คะแนนของผู้ใช้ u ต่อรายการ i

4 ตัวอย่างการคำนวณอย่างง่าย

กำหนดเวกเตอร์ขนาด 2 มิติ สำหรับผู้ใช้ 1 คน และรายการภาพยนตร์ 1 เรื่อง ที่มี 2 ประเภท (animation และ family)

ส่วนประกอบ	สัญลักษณ์	เวกเตอร์ (2 มิติ)	คำอธิบาย
เวกเตอร์ผู้ใช้	p_u	[0.8, 0.2]	ความชอบของผู้ใช้ u
เวกเตอร์รหัสรายการ	e_i^{id}	[0.3, 0.1]	รหัสของรายการ i
เวกเตอร์ประเภท (animation)	$e_{animation}$	[0.2, 0.0]	คุณลักษณะประเภท animation
เวกเตอร์ประเภท (family)	e_{family}	[0.1, 0.2]	คุณลักษณะประเภท family

ขั้นตอนการคำนวณ

1. **รวมเวกเตอร์ของรายการ**

$$q_i = [0.3, 0.1] + [0.2, 0.0] + [0.1, 0.2] = [0.6, 0.3]$$

2. **คำนวณคะแนน (ผลคูณจุด)**

$$\hat{y}(u,i) = [0.8, 0.2] \cdot [0.6, 0.3] = 0.8 \times 0.6 + 0.2 \times 0.3 = 0.48 + 0.06 = \mathbf{0.54}$$

ดังนั้น รายการภาพยนตร์เรื่องนี้ได้รับคะแนน 0.54 สำหรับผู้ใช้ u

5 สรุปสั้น ๆ

- ใช้ข้อมูลหลายแหล่งพร้อมกันตั้งแต่ขั้นตอนนำเข้า
- เหมาะกับปัญหา cold-start มากกว่า MF แบบดั้งเดิม เพราะยังใช้ข้อมูลเนื้อหา/คุณลักษณะได้
- ตัวอย่างที่นิยมคือ LightFM (Early Fusion)
- สามารถเรียนรู้ความสัมพันธ์ระหว่างผู้ใช้ รายการ และเนื้อหาได้ในโมเดลเดียว

ข้อควรระวัง
- ต้องออกแบบและเลือกคุณลักษณะ (feature) ให้เหมาะสม
- คุณภาพของข้อมูลเนื้อหา (เช่น ประเภท, ข้อความ) มีผลมาก
- ต้องฝึกโมเดลเดียวให้สมดุลระหว่างข้อมูลหลายแบบ
- หากข้อมูลบางส่วนมีคุณภาพต่ำ อาจทำให้ประสิทธิภาพโดยรวมลดลง

1 LightFM โมเดลผสมปัจจัยแฝงกับคุณลักษณะ (Hybrid MF with Features)

แนวคิด (Idea)

LightFM รวมปัจจัยแฝงของผู้ใช้/รายการ กับคุณลักษณะ (feature) เช่น แนวภาพยนตร์ ผู้กำกับ หรือข้อมูลผู้ใช้ (เพศ อายุ) เพื่อแก้ปัญหาข้อมูลเบาบางและแนะนำรายการใหม่ได้



สูตรให้คะแนน

$$\hat{r}_{ui} = \left(p_u + \sum_{f \in F_u} p_f\right)^T \left(q_i + \sum_{g \in F_i} q_g\right)$$

p_u, q_i : เวกเตอร์ปัจจัยแฝงของผู้ใช้/รายการ
p_u, q_g : เวกเตอร์ของคุณลักษณะ (feature)
F_u, F_i : ชุดคุณลักษณะของผู้ใช้และรายการ

ขั้นตอนการคำนวณ

1. รวมเวกเตอร์ของผู้ใช้และคุณลักษณะผู้ใช้
$$x_u = p_u + \sum p_f$$
2. รวมเวกเตอร์ของรายการและคุณลักษณะรายการ
$$y_i = q_i + \sum q_g$$
3. คำนวณผลคูณจุด
$$\hat{r}_{ui} = x_u^T y_i$$

ตัวอย่างการคำนวณ (แบบง่าย)

ข้อมูลตัวอย่าง (2 มิติ)

p_u (u1)	= [0.3, 0.1]	
p_{female}	= [0.1, 0.0]	u1 เป็นผู้หญิง
q_i (i2)	= [0.2, 0.4]	-
$q_{animation}$	= [0.1, 0.1]	i2 แนว Animation

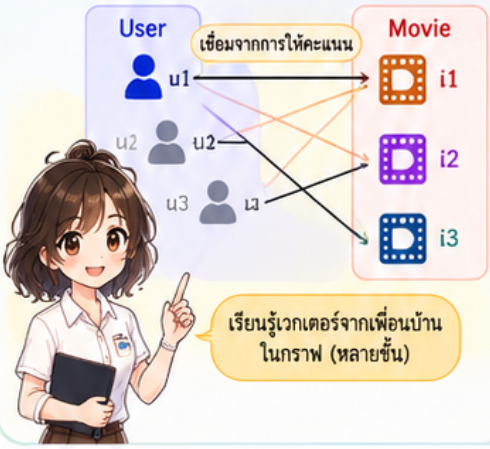
1. รวมเวกเตอร์
$$x_u = [0.3, 0.1] + [0.1, 0.0] = [0.4, 0.1]$$
$$y_i = [0.2, 0.4] + [0.1, 0.1] = [0.3, 0.5]$$
2. ผลคูณจุด
$$\hat{r}_{u1,i2} = 0.4 \times 0.3 + 0.1 \times 0.5 = 0.12 + 0.05 = 0.17$$

สรุป
คะแนนความชอบของ u1 ต่อ i2 คือ 0.17 (ยิ่งสูงยิ่งน่าจะแนะนำ)

2 LightGCN โมเดลกราฟแบบเบาเพื่อการแนะนำ (Graph-based Collaborative Filtering)

แนวคิด (Idea)

LightGCN มองผู้ใช้และรายการเป็นโหนดในกราฟความสัมพันธ์ (User–Item Graph) และเรียนรู้เวกเตอร์โดยการเผยแพร่ (propagation) เฉพาะเพื่อนบ้าน โดยใช้เพียงโครงสร้างกราฟ (ไม่ใช้พารามิเตอร์การแปลงเชิงซ้อน)



สูตรให้คะแนน

$$\hat{r}_{ui} = e_u^T e_i$$

e_u, e_i : เวกเตอร์ปัจจัยและรายการหลังการเผยแพร่ L ชั้น

$$e_u = \frac{1}{L+1} \sum_{l=0}^{L} e_u^{(l)} \quad \text{(และ } e_i \text{ เช่นเดียวกัน)}$$

ขั้นตอนการคำนวณ (L = 1)

1. เฉลี่ยเวกเตอร์ของเพื่อนบ้าน
$$e_u^{(1)} = \frac{1}{|N(u)|} \sum_{i \in N(u)} e_i^{(0)}$$
$$e_i^{(1)} = \frac{1}{|N(i)|} \sum_{u \in N(i)} e_u^{(0)}$$
2. รวมชั้น (L = 1)
$$e_u = \frac{e_u^{(0)} + e_u^{(1)}}{2}, \quad e_i = \frac{e_i^{(0)} + e_i^{(1)}}{2}$$
3. คำนวณผลคูณจุด $\hat{r}_{ui} = e_u^T e_i$

ตัวอย่างการคำนวณ (แบบง่าย) : ทำนาย u1–i3

ข้อมูลเริ่มต้น (2 มิติ)

$e_{u1}^{(0)} = [1.0, 0.0]$	(u1 เคยให้คะแนน i1, i2)
$e_{i1}^{(0)} = [1.0, 0.0]$	-
$e_{i2}^{(0)} = [0.0, 1.0]$	-
$e_{i3}^{(0)} = [0.5, 0.5]$	(ยังไม่มีเพื่อนบ้าน)

1. คำนวณ $e_{u1}^{(1)}$ จากเพื่อนบ้าน i1, i2
$$e_{u1}^{(1)} = \frac{[1.0,0.0] + [0.0,1.0]}{2} = [0.5, 0.5]$$
2. รวมชั้น (L = 1)
$$e_{u1} = \frac{[1.0,0.0] + [0.5,0.5]}{2} = [0.75, 0.25]$$
$$e_{i3} = \frac{[0.5,0.5] + [0.0,0.0]}{2} = [0.25, 0.25]$$
3. ผลคูณจุด
$$\hat{r}_{u1,i3} = 0.75 \times 0.25 + 0.25 \times 0.25 = 0.1875 + 0.0625 = 0.25$$

สรุป
คะแนนความชอบของ u1 ต่อ i2 คือ 0.25

3 logQ การลดอคติรายการยอดนิยมด้วยการปรับน้ำหนักด้วย log (Popularity Debiasing)

แนวคิด (Idea)

logQ ปรับคะแนนของรายการที่พยากรณ์ได้ด้วยปัจจัยลดทอนตามความนิยม เพื่อไม่ให้รายการยอดนิยมได้เปรียบมากเกินไป โดยใช้ log ของจำนวนการมีปฏิสัมพันธ์ (เช่น จำนวนผู้ใช้ที่ให้คะแนน)



สูตรให้คะแนน

$$score'(u,i) = \frac{score(u,i)}{\log(1 + Q(i))}$$

score(u,i) : คะแนนที่ได้จากโมเดลฐาน (เช่น MF, kNN, NCF)
$Q(i)$: จำนวนผู้ใช้ที่ให้คะแนนหรือมีปฏิสัมพันธ์กับรายการ i
log ช่วยลดทอนผลของรายการที่มีผู้ใช้จำนวนมาก

ขั้นตอนการคำนวณ

1. คำนวณคะแนนจากโมเดลฐาน $score(u,i)$
2. นับจำนวนผู้ใช้ของรายการ $Q(i)$
3. ปรับด้วย log $score'(u,i) = \frac{score(u,i)}{\log(1 + Q(i))}$

ตัวอย่างการคำนวณ (แบบง่าย)

ข้อมูลตัวอย่าง

รายการ	คะแนนจากโมเดลฐาน score(u,i)	จำนวนผู้ใช้ Q(i)
i1 (ยอดนิยม)	0.80	100
i2 (ปานกลาง)	0.60	10
i3 (ไม่ค่อยนิยม)	0.40	2

คำนวณคะแนนหลังปรับ logQ

i1: 0.80 / log(1+100) = 0.80 / 4.615 = 0.173
i2: 0.60 / log(1+10) = 0.60 / 2.398 = 0.250
i3: 0.40 / log(1+2) = 0.40 / 1.099 = 0.364

สรุป
หลังปรับด้วย logQ รายการที่ไม่ค่อยนิยม (i3) มีคะแนนสูงขึ้น เมื่อเทียบกับรายการยอดนิยม (i1)

Feature Augmentation Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

หนึ่งใน 7 รูปแบบของ Burke

1 Feature Augmentation คืออะไร?

Feature Augmentation เป็นวิธีผสานผลลัพธ์จากระบบหนึ่งมาเป็น **"ข้อมูลเพิ่ม"** (feature หรือคะแนนเทียม) แล้วนำข้อมูลนั้นไปใช้เป็นอินพุตของอีกระบบหนึ่ง โดยระบบแรกจะสร้างคุณลักษณะเพิ่มเติมหรือค่าคาดการณ์ให้กับข้อมูลเดิม จากนั้นระบบที่สองจะใช้ข้อมูลที่เพิ่มขึ้นนี้ในการแนะนำรายการ

ตัวอย่างในบทนี้ ใช้ระบบแนะนำที่อิงข้อมูลเนื้อหา (Text-based หรือ Content-based) ในการคาดการณ์คะแนนของรายการที่ยังไม่มีการให้คะแนน (เติมค่าที่หายไป) ก่อน แล้วจึงนำเมทริกซ์ที่หนาแน่นขึ้นไปใช้กับระบบ User-kNN เพื่อทำนายคะแนนและแนะนำรายการ

2 แนวคิดหลัก

Model A (Content-based) → คาดการณ์คะแนนจากข้อมูลเนื้อหา

→ **เติมคะแนนเทียมหรือ Feature เพิ่ม** → คาดการณ์คะแนนรายการที่ยังไม่มีการให้คะแนน

→ **เมทริกซ์หนาแน่นขึ้น** → ได้เมทริกซ์ที่มีข้อมูลมากขึ้น ลดปัญหาข้อมูลเบาบาง

→ **Model B (User-kNN)** → ใช้ข้อมูลที่เพิ่มขึ้นในการคำนวณความคล้ายคลึงและทำนายคะแนน

→ **จัดอันดับรายการตามคะแนนสุดท้าย**

3 วิธีคิดสำคัญ

ส่งต่อผลลัพธ์ของ Model A เป็น input เพิ่มให้ Model B

คะแนนเทียมที่เพิ่มเข้ามาช่วยให้การคำนวณความคล้ายคลึงระหว่างผู้ใช้ (similarity) ของ User-kNN มีข้อมูลมากขึ้นและแม่นยำขึ้น

สูตรทำนายคะแนนของ User-kNN (ขั้นตอนที่สอง)

$$\hat{y}(u,i) = \bar{r}_u + \frac{\sum_{v \in N(u)} sim(u,v)(r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |sim(u,v)|}$$

- $\hat{y}(u,i)$ = คะแนนที่คาดการณ์ของผู้ใช้ u สำหรับรายการ i
- $\bar{r}_u$ = ค่าเฉลี่ยคะแนนของผู้ใช้ u
- $sim(u,v)$ = ค่าความคล้ายคลึงระหว่างผู้ใช้ u และ v
- $r_{v,i}$ = คะแนนที่ผู้ใช้ v ให้กับรายการ i
- $\bar{r}_v$ = ค่าเฉลี่ยคะแนนของผู้ใช้ v
- $N(u)$ = กลุ่มเพื่อนบ้านของผู้ใช้ u

4 ตัวอย่างการคำนวณอย่างง่าย

ขั้นตอนที่ 1 เติมคะแนนเทียมด้วย Content-based

ผู้ใช้ U1 ให้คะแนนรายการ i1 = 5 และ i3 = 4 แต่ยังไม่มีคะแนน i2 ระบบ Content-based คาดการณ์คะแนนของ i2 ได้เป็น 4.2 ทำให้เมทริกซ์มีข้อมูลมากขึ้น

ผู้ใช้	i1	i2	i3	...
U1	5	4.2 (คาดการณ์)	4	...

ขั้นตอนที่ 2 ใช้ User-kNN ทำนายคะแนนรายการเป้าหมาย

ทำนายคะแนนของผู้ใช้ U1 สำหรับรายการ i5 โดยใช้เพื่อนบ้าน U4 และ U7 โดยมีข้อมูลดังนี้

ผู้ใช้ (v)	sim(U1, v)	คะแนน $r_{v,5}$	ค่าเฉลี่ย $\bar{r}_v$	$(r_{v,5} - \bar{r}_v)$
U4	0.80	5	4.0	1.0
U7	0.40	4	3.5	0.5

แทนค่าในสูตร

$$\hat{y}(1,5) = 2.75 + \frac{0.8 \times 1.0 + 0.4 \times 0.5}{0.8 + 0.4}$$

$$= 2.75 + \frac{0.8 + 0.2}{1.2} = 2.75 + 0.833 \approx 3.58$$

โดยค่าเฉลี่ยคะแนนของผู้ใช้ U1 คือ $\bar{r}_{U1} = 2.75$

ดังนั้น ระบบที่มีการเติมคะแนนเทียมจะทำนายคะแนนของ U1 สำหรับรายการ i5 ได้ประมาณ 3.58

5 สรุปสั้น ๆ

- ช่วยแก้ปัญหาข้อมูลเบาบาง (sparsity) ได้ดี
- ใช้จุดแข็งของระบบแรกช่วยเสริมข้อมูลให้ระบบที่สอง
- ทำให้การคำนวณความคล้ายคลึงและการทำนายคะแนนแม่นยำขึ้น
- เหมาะสมกับสถานการณ์ที่มีข้อมูลรายการจำนวนมาก แต่ยังมีการให้คะแนนน้อย

ข้อควรระวัง

- ถ้าค่าคะแนนเทียมที่เติมจากระบบแรกผิดพลาดมาก อาจส่งผลเสียต่อระบบที่สองได้
- คุณภาพของการแนะนำขึ้นอยู่กับความแม่นยำของระบบแรก
- ควรเลือกวิธีคาดการณ์ที่เหมาะสมและตรวจสอบผลลัพธ์ เพื่อลดผลกระทบจากค่าที่ไม่ถูกต้อง

Cascade Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

หนึ่งใน 7 รูปแบบของ Burke

1 Cascade Hybrid คืออะไร?

Cascade Hybrid เป็นวิธีผสานผลลัพธ์จากตัวแนะนำหลายระบบแบบ**เป็นลำดับขั้น** โดยให้ระบบแรกทำหน้าที่คัดกรองหรือจำกัดจำนวนไอเท็มผู้สมัคร (candidate items) จากทั้งหมดก่อน แล้วจึงให้ระบบที่สองทำหน้าที่จัดอันดับอย่างละเอียดเฉพาะไอเท็มที่ผ่านการคัดกรองมาเท่านั้น โดยแต่ละระบบมีหน้าที่รับผิดชอบต่างกัน

มักใช้เมื่อระบบหนึ่งทำงานได้รวดเร็ว เหมาะสำหรับการคัดกรองจำนวนมาก และอีกระบบหนึ่งมีความแม่นย่าสูง เหมาะสำหรับการจัดอันดับอย่างละเอียด

2 แนวคิดหลัก

ขั้นตอนที่ 1
คัดกรองผู้สมัคร (Model A)

เช่น ระบบ Content-based ที่ทำงานได้รวดเร็ว เพื่อเลือกไอเท็มที่เกี่ยวข้อง

ไอเท็มทั้งหมด (เช่น 10,000 รายการ)

เหลือเฉพาะไอเท็มผู้สมัคร (Top-M จาก Model A)



ขั้นตอนที่ 2
จัดอันดับละเอียด (Model B)

เช่น ระบบ User-kNN ที่มีความแม่นยำสูง เพื่อจัดอันดับเฉพาะไอเท็มที่เหลือ

ผลลัพธ์สุดท้าย (Top-N)



3 วิธีคิดสำคัญ

ใช้ผลของระบบแรกเป็นข้อจำกัดของระบบที่สอง

$$Final\ list = rerank(Top\text{-}M_A, using\ Model\ B)$$

ช่วยลดเวลาในการคำนวณ เพราะไม่ต้องจัดอันดับทุกไอเท็มด้วยโมเดลที่มีความซับซ้อน (เช่น User-kNN) แต่ทำเฉพาะไอเท็มที่ผ่านการคัดกรองมาแล้ว

$Top\text{-}M_A$	=	ไอเท็ม M รายการแรกที่ได้จากระบบแรก (Model A)
$rerank(\cdot)$	=	การจัดอันดับใหม่โดยใช้ระบบที่สอง (Model B) เฉพาะไอเท็มที่เหลือ
$Top\text{-}N$	=	รายการแนะนำสุดท้าย จำนวน N รายการ

4 ตัวอย่างการคำนวณอย่างง่าย

สมมติว่ามีไอเท็มทั้งหมด 4 รายการ โดยใช้ระบบ 2 ขั้นตอน
ขั้นตอนที่ 1 ใช้ Content-based คัดกรองเหลือ 3 รายการ และขั้นตอนที่ 2 ใช้ User-kNN จัดอันดับใหม่

ขั้นตอนที่ 1: Content-based คัดกรองผู้สมัคร (Model A)

รายการ	คะแนน$_{CB}$	ผลการคัดกรอง
i2	0.92	✓ เลือก (Top-3)
i7	0.88	✓ เลือก (Top-3)
i9	0.75	✓ เลือก (Top-3)
i10	0.60	✗ ตัดออก

ขั้นตอนที่ 2: User-kNN จัดอันดับเฉพาะผู้สมัคร (Model B)

รายการ (เฉพาะที่ผ่าน)	คะแนน$_{kNN}$	ผลการจัดอันดับ
i2	4.4	-
i7	4.0	-
i9	4.8	-

หมายเหตุ: i10 ถูกตัดออกในขั้นตอนที่ 1 จึงไม่ถูกนำมาจัดอันดับในขั้นตอนที่ 2

ผลการจัดอันดับสุดท้าย (Top-3)

อันดับ	รายการ	คะแนน$_{kNN}$
1	i9	4.8
2	i2	4.4
3	i7	4.0

5 สรุปสั้น ๆ

จุดเด่น

- ชัดเจนว่าแต่ละระบบทำหน้าที่ต่างกัน (คัดกรอง และ จัดอันดับ)
- ประหยัดเวลาในการคำนวณ เพราะไม่ต้องจัดอันดับทุกไอเท็มด้วยโมเดลที่มีความซับซ้อน
- เหมาะสำหรับสถานการณ์ที่มีไอเท็มจำนวนมาก

ข้อควรระวัง

- หากระบบแรกคัดกรองผิดพลาด ไอเท็มที่ดีอาจถูกตัดทิ้งตั้งแต่ต้น และจะไม่มีโอกาสให้ระบบที่สองพิจารณา
- คุณภาพของผลลัพธ์ขึ้นอยู่กับทั้งสองระบบ
- ควรเลือกวิธีและเกณฑ์การคัดกรองของระบบแรกให้เหมาะสม เพื่อลดความเสี่ยงในการตัดไอเท็มดีทิ้ง

Meta-level Hybrid (Burke) Recommendation Model

แนวคิด + ตัวอย่างการคำนวณอย่างง่าย

หนึ่งใน 7 รูปแบบของ Burke

1 Meta-level คืออะไร?

Meta-level Hybrid เป็นวิธีผสานผลลัพธ์ที่ไม่ใช่แค่ **"คะแนน (score)"** เท่านั้น แต่เป็นการส่งต่อ **"โมเดลหรือข้อมูลตัวแทน (model/representation)"** จากระบบหนึ่งไปยังอีกระบบหนึ่ง

ตัวอย่างจากบทนี้: สร้างโปรไฟล์ข้อความของผู้ใช้จากข้อมูลเนื้อหา (เช่น รีวิว รายการที่ชอบ) แล้วใช้โปรไฟล์นี้เป็นพื้นฐานในการหาความคล้ายของผู้ใช้ในระบบ User-kNN แทนการใช้ความคล้ายจากรายการที่ให้คะแนนร่วมกัน

2 แนวคิดหลัก

สร้างโปรไฟล์ผู้ใช้จากข้อความ (เช่น TF-IDF, Vector) → **ใช้โปรไฟล์เป็นตัวแทนของผู้ใช้** (Representation) → **คำนวณความคล้ายระหว่างผู้ใช้** (เช่น Cosine Similarity) จากโปรไฟล์ → **ส่งเข้า User-kNN** (หาผู้ใช้ที่คล้ายกัน) → **ทำนายคะแนน** สำหรับรายการที่สนใจ

3 วิธีคิดสำคัญ

1) ความคล้ายของผู้ใช้จากโปรไฟล์ (Cosine Similarity)

$$\cos(a, b) = \frac{a \cdot b}{\|a\| \times \|b\|}$$

- **a, b** = เวกเตอร์โปรไฟล์ของผู้ใช้ 2 คน
- **a·b** = ผลคูณเชิงสเกลาร์ของเวกเตอร์
- **‖a‖** = ความยาวของเวกเตอร์

2) การทำนายคะแนนด้วย User-kNN (โดยใช้ความคล้ายจากโปรไฟล์)

$$\hat{y}(u, i) = \bar{r}_u + \frac{\sum_{v \in N(u)} sim(u, v) \cdot (r_{v,i} - \bar{r}_v)}{\sum_{v \in N(u)} |sim(u, v)|}$$

- $sim(u, v)$ = ความคล้ายของผู้ใช้จากโปรไฟล์ (ไม่ใช้จากรายการที่ให้คะแนนร่วมกัน)
- $r_{v,i}$ = คะแนนที่ผู้ใช้ v ให้กับรายการ i
- $\bar{r}_u, \bar{r}_v$ = คะแนนเฉลี่ยของผู้ใช้ u และ v
- $N(u)$ = เซตของเพื่อนบ้านของผู้ใช้ u (เช่น เลือกผู้ใช้ที่คล้ายกันมากที่สุด k คน)

4 ตัวอย่างการคำนวณอย่างง่าย

สมมติว่าเรามีโปรไฟล์ข้อความของผู้ใช้ U1, U3 และ U4 และต้องการทำนายคะแนนของรายการ i2 สำหรับผู้ใช้ U1 โดยใช้ User-kNN ที่มีเพื่อนบ้าน 2 คน (U3 และ U4)

1) โปรไฟล์ของผู้ใช้ (เวกเตอร์จากข้อความ)

ผู้ใช้	โปรไฟล์ (p)
U1	[1.0, 0.5]
U3	[0.8, 0.2]
U4	(ไม่แสดงค่า)

2) คำนวณความคล้ายระหว่าง U1 และ U3

$$\cos(U_1, U_3) = \frac{1.0 \times 0.8 + 0.5 \times 0.2}{\sqrt{1.0^2 + 0.5^2} \times \sqrt{0.8^2 + 0.2^2}} = \frac{0.9}{1.118 \times 0.825} \approx \mathbf{0.98}$$

สมมติว่าความคล้ายระหว่าง U1 และ U4 (จากโปรไฟล์) = 0.77

3) ข้อมูลคะแนนของเพื่อนบ้าน

ผู้ใช้	คะแนนที่ให้ i2	คะแนนเฉลี่ย ($\bar{r}$)
U3	5	4.0
U4	4	3.5

รายการ	$sim(U_1, v)$	$(r_{v,i2} - \bar{r}_v)$	คูณกับความคล้าย
U3	0.98	5 − 4.0 = 1.0	0.98 × 1.0 = 0.98
U4	0.77	4 − 3.5 = 0.5	0.77 × 0.5 = 0.385

4) คำนวณคะแนนที่ทำนายสำหรับ U1, i2

$$\hat{y}(U_1, i_2) = 3.2 + \frac{0.98 \times 1.0 + 0.77 \times 0.5}{0.98 + 0.77} = 3.2 + \frac{0.98 + 0.385}{1.75} = 3.2 + 0.78 \approx \mathbf{3.98}$$

โดยที่ คะแนนเฉลี่ยของ U1 ($\bar{r}_{U_1}$) = 3.2

5 สรุปสั้น ๆ

จุดเด่น

- ช่วยส่งต่อความรู้ระดับโมเดลหรือข้อมูลตัวแทนระหว่างระบบ
- ลดปัญหาการหาเพื่อนบ้านไม่ได้เมื่อไม่มีรายการที่ให้คะแนนร่วมกัน
- สามารถใช้ข้อมูลเนื้อหาเพื่อสร้างความคล้าย ที่มีความหมายมากขึ้น
- เหมาะกับสถานการณ์ที่ข้อมูลคะแนนมีความเบาบาง (sparse)

ข้อควรระวัง

- โปรไฟล์ที่ส่งต่ออาจสะท้อนลักษณะเนื้อหามากกว่าพฤติกรรมการให้คะแนนจริง
- คุณภาพของผลลัพธ์ขึ้นอยู่กับวิธีการสร้างโปรไฟล์และคุณภาพของข้อมูลเนื้อหา
- อาจไม่เหมาะกับโดเมนที่ข้อมูลเนื้อหาไม่เพียงพอ หรือโปรไฟล์ไม่สามารถแทนความสนใจของผู้ใช้ได้อย่างแท้จริง